microback

C++20 Market Microstructure & Backtesting Engine
Technical Specification

Purpose Build a portfolio-grade research/trading-systems project that demonstrates C++ systems depth, market microstructure understanding, realistic execution modelling, quantitative research hygiene, and performance engineering - not merely a profitable-looking backtest.

Attribute	Specification
Primary goal	Quant Developer / HFT Software Engineer differentiation
Core	C++20 deterministic event-driven simulator
Research layer	Python via pybind11; pandas/NumPy/Matplotlib
Market model	Single-venue, single-instrument limit order book; L2 first, L3 optional
Execution realism	Latency, queue position, partial fills, fees, spread, slippage, adverse selection
Research correctness	No look-ahead; train/test and walk-forward evaluation
Engineering	Unit/integration tests, sanitizers, benchmarks, profiling, CI, documented architecture
Performance	Measure before optimising; report reproducible event throughput and hot-path profiles

Differentiation thesis

- Use order-level or depth market events rather than OHLC/candle backtests.
- Keep the simulation core in C++ and the research surface in Python.
- Model uncertainty explicitly: L2 queue position must be an assumption, not fake precision.
- Treat causality, latency and execution as first-class constraints.
- Generate measurable systems evidence: throughput, cache/branch metrics, profiler findings and justified optimisations.

1. Scope, goals and non-goals

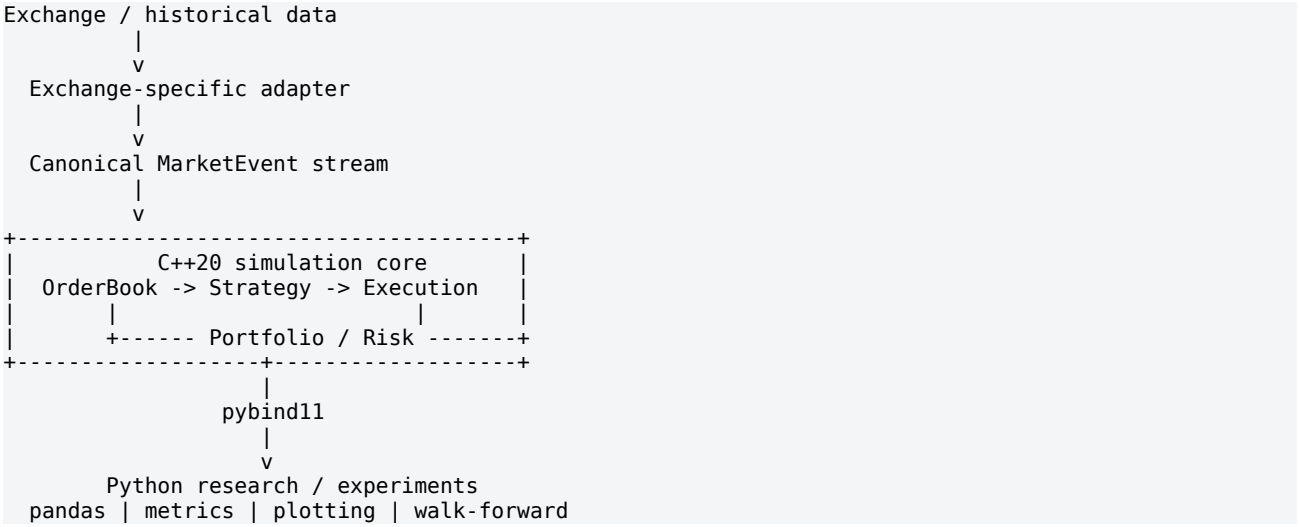
Area	v1 requirement
Venue	One exchange / one venue
Instrument	One instrument per engine instance
Asset class	Crypto initially for accessible event data
Book	Limit order book
Market data	L2 required; L3 optional extension
Order types	Market + Limit
Time in force	GTC + IOC
Fills	Partial fills supported
Costs	Maker/taker fees + measured slippage
Latency	Configurable
Queue	Modelled for L2; exact only where L3 data permits
Strategies	C++ native; Python research interface
Validation	Walk-forward evaluation + deterministic tests
Concurrency	Across independent runs, not forced into one event stream

Non-goals for v1

- Live-money trading
- Multi-venue smart order routing
- Options/derivatives pricing
- GPU acceleration
- Distributed microservices
- React dashboard / REST API / authentication
- Cloud deployment as a core deliverable

Design rule Depth beats breadth. Do not add infrastructure that does not improve market simulation fidelity, research correctness or measurable performance.

2. Architecture



Architectural invariants

- Exchange-specific parsing never leaks into the simulation core.

- Strategy code can read market/portfolio state and submit actions, but cannot mutate the order book directly.
- The event loop is deterministic for identical data, configuration, seed and strategy.
- Market simulation remains single-threaded per instrument; batch experiments may run in parallel.
- Hot-path market types use fixed-width integers and compact enums; no strings or floating-point prices in the core.

3. Core domain model

3.1 Integer representation

Use integer ticks/lots throughout the simulation. Convert to decimal units only at API/reporting boundaries.

```
using Timestamp    = std::int64_t;   // nanoseconds
using PriceTicks   = std::int64_t;
using QuantityLots = std::int64_t;
using OrderId      = std::uint64_t;
using Sequence     = std::uint64_t;
```

Reason Integer prices/lots preserve equality, deterministic replay and exact book/accounting semantics while avoiding floating-point drift.

3.2 Required enums and records

```
enum class Side : uint8_t { Buy, Sell };
enum class MarketEventType : uint8_t { Snapshot, Add, Modify, Cancel, Trade };
enum class OrderType : uint8_t { Market, Limit };
enum class TimeInForce : uint8_t { GTC, IOC };
enum class OrderStatus : uint8_t { Pending, Active, PartiallyFilled, Filled, Cancelled, Rejected };

struct MarketEvent {
    Timestamp timestamp_ns;
    Sequence sequence;
    OrderId order_id;
    PriceTicks price;
    QuantityLots quantity;
    MarketEventType type;
    Side side;
};
```

3.3 Simulated order

```
struct SimOrder {
    uint64_t client_order_id;
    Side side;
    OrderType type;
    TimeInForce tif;
    PriceTicks limit_price;
    QuantityLots initial_quantity;
    QuantityLots remaining_quantity;
    Timestamp submitted_at;
    Timestamp exchange_arrival_at;
    OrderStatus status;
};
```

4. Limit order book

Implement price-time priority. Within each price level, orders are FIFO. Maintain an order-id index so cancels/modifies do not require full-book scans.

```
std::map<PriceTicks, PriceLevel, std::greater<>> bids;
std::map<PriceTicks, PriceLevel, std::less<>>    asks;
std::unordered_map<OrderId, OrderLocation>       order_index;

struct PriceLevel {
    QuantityLots total_quantity;
    std::list<Order> orders; // simple/stable baseline; profile later
};
```

Operation	Target behaviour
Best bid/ask	O(1) from map begin()
Price-level insert/lookup	O(log P), P = number of levels

Operation	Target behaviour
Order lookup	Average O(1) via order_index
Cancel	Locate indexed order, erase, update level aggregate, remove empty level
Modify price	Equivalent to cancel + reinsert; queue priority resets
Modify quantity	Define exchange-consistent semantics; increases should lose priority unless source data says otherwise

Required OrderBook API

```
void add(const AddOrder&);
void cancel(OrderId);
void modify(const ModifyOrder&);
void apply(const MarketEvent&);

PriceTicks best_bid() const;
PriceTicks best_ask() const;
PriceTicks spread_ticks() const;
PriceTicks mid_price() const;
PriceTicks microprice() const;
double imbalance(std::size_t levels) const;
DepthView depth(Side, std::size_t levels) const;
PriceTicks vwap(Side, QuantityLots) const;
bool valid() const;
```

5. Market microstructure state

Metric	Definition / requirement
Mid-price	(best_bid + best_ask) / 2; retain half-tick precision in reporting if needed
Spread	best_ask - best_bid, in ticks
Top-N imbalance	(bid_volume - ask_volume) / (bid_volume + ask_volume) over N levels
Microprice	Opposite-side quantity weighted best bid/ask
Depth	Price/quantity levels by side
VWAP	Average visible execution price required to consume a specified quantity

Microprice

```
microprice =
    (ask_qty * best_bid + bid_qty * best_ask)
    / (bid_qty + ask_qty)
```

6. Deterministic event replay and causality

6.1 Ordering

Process external market events in lexicographic (timestamp_ns, sequence) order. Reject or explicitly reset on invalid timestamp/sequence regressions.

```
struct ScheduledEvent {
    Timestamp timestamp;
    uint64_t internal_sequence;
    EventKind kind;
};
```

6.2 Mandatory event-cycle semantics

Step	Action
1	Apply the external market event to the reconstructed book.
2	Update derived market state.
3	Process any exchange/fill effects caused by scheduled actions.
4	Invoke the strategy callback with state observable at time t.
5	Accept strategy submit/cancel requests.
6	Schedule each action for t + sampled/configured latency.
7	Process the action only when its exchange-arrival event becomes due.

No-look-ahead invariant An action generated after observing event t can never receive a fill using book state that existed before that action reached the simulated exchange.

6.3 Determinism

Identical input data + configuration + random seed + strategy must produce identical order/fill/PnL results. Add a regression test that compares two full runs.

7. Execution model

7.1 Latency

```
class LatencyModel {
public:
    virtual Timestamp sample_latency() = 0;
};

// v1: ConstantLatency
// optional: UniformLatency, LognormalLatency
```

A strategy decision at time t becomes actionable only at t + latency. Market movement during that interval changes execution outcomes.

7.2 Market orders

Walk visible liquidity across price levels. Support partial fills. Never fill an arbitrary size entirely at the top-of-book price.

7.3 Crossing limit orders

A buy limit at/above best ask or sell limit at/below best bid takes liquidity up to its limit price and incurs taker economics.

7.4 Passive limit orders - L2

Queue assumption	Cancellation treatment
Conservative (default)	Cancellations do not reduce volume ahead.
Neutral	Reduce volume ahead proportionally to observed cancellation.
Optimistic	Assume cancellations remove volume ahead first.

Initial queue-ahead is visible quantity already resting at that price. Only executions/cancellation assumptions can move the simulated order forward. This is explicitly an approximation because L2 data cannot reveal true queue position.

7.5 L3 extension

When individual order events are available, maintain actual order identity and FIFO queue position. Expose ExecutionMode::L2Approximation and ExecutionMode::L3ExactQueue rather than silently changing semantics.

8. Costs, slippage, portfolio and risk

8.1 Fees and rebates

```
fee = notional * bps / 10,000
```

Track gross trading PnL, maker/taker fees or rebates, and net PnL separately.

8.2 Slippage decomposition

Stored reference	Use
Decision mid	Mid-price visible when strategy decides.
Arrival mid	Mid-price when order reaches simulated exchange.
Execution price	Actual simulated fill price.

For buys: decision slippage = execution - decision_mid; arrival slippage = execution - arrival_mid. Reverse sign for sells. This separates latency/signal decay from execution/book-walking cost.

8.3 Adverse selection

For every fill, record mid-price after fixed horizons (e.g. 1 ms, 10 ms, 100 ms, 1 s) and calculate signed post-fill movement. Compute after simulation; never expose future values to strategies.

8.4 Portfolio

- cash / inventory
- average entry price
- realised and unrealised PnL
- gross and net PnL
- fees/rebates
- turnover and gross exposure

Mark open inventory at mid-price by default.

8.5 Risk controls

```
struct RiskLimits {
    QuantityLots max_position;
    QuantityLots max_order_size;
    double max_notional;
};
```

Reject and log orders that violate configured risk limits. Keep risk checks on the order-submission path.

9. Strategy interface and reference strategies

```
class Strategy {
public:
    virtual ~Strategy() = default;
    virtual void on_start(Context&) {}
    virtual void on_market_event(Context&, const MarketEvent&) = 0;
    virtual void on_fill(Context&, const Fill&) {}
    virtual void on_timer(Context&, Timestamp) {}
    virtual void on_end(Context&) {}
};
```

Context provides read-only access to current market/portfolio state and controlled submit_limit, submit_market and cancel operations.

9.1 Reference strategy A - order-book imbalance

Use top-N depth imbalance as a short-horizon signal. Example default: N=5; enter above +0.35 / below -0.35; flatten near 0.10. Parameters must be experiment configuration, not hard-coded.

9.2 Reference strategy B - inventory-skewed market maker

Quote passively around the spread and shift reservation price/quote aggressiveness against inventory. Use this primarily to stress queue modelling, cancellations, latency, fill quality, adverse selection and risk limits.

Research principle The reference strategies exist to validate the engine and study execution effects. Profitability is not the success criterion.

10. Data architecture and validation

10.1 EventSource boundary

```
class EventSource {
public:
    virtual bool next(MarketEvent& event) = 0;
};

// CsvEventSource
// BinaryEventSource
// SyntheticEventSource
```

Exchange-specific JSON/CSV/websocket payloads are converted into the canonical event schema before simulation.

10.2 Development data modes

Mode	Purpose
Tiny synthetic fixtures	Exact unit/integration expectations and edge cases.
Large synthetic stream	1M/10M/100M event performance benchmarks.
Real exchange data	Research realism and final demonstration. Start with one liquid crypto instrument; Kraken-style L2/L3 data is a practical target.

10.3 Binary replay format

```
struct MarketEventRecord {
    uint64_t timestamp_ns;
    uint64_t sequence;
    uint64_t order_id;
    int64_t  price_ticks;
    int64_t  quantity_lots;
    uint8_t  type;
    uint8_t  side;
    uint16_t flags;
    uint32_t symbol_id;
};
static_assert(std::is_trivially_copyable_v<MarketEventRecord>);
```

Ship a converter CLI and benchmark CSV replay vs binary replay. Keep raw market datasets out of Git.

10.4 Validation before replay

- monotonic timestamps / valid sequence handling
- non-negative quantities
- tick-aligned prices
- snapshot before incremental updates where required
- known order IDs for L3 cancels/modifies
- crossed-book/invariant detection
- counters for dropped events, sequence gaps and resets

Fail fast by default. A lenient exploration mode may log and continue, but must be opt-in.

11. Python research layer and pybind11

11.1 Binding surface

Expose stable research primitives rather than every internal implementation type. The extension module should be microback._core; the public package wraps it with Pythonic configuration and results.

```
result = microback.run(
    data="BTCUSD.mkt",
    strategy="imbalance",
    parameters={"levels": 5, "entry_threshold": 0.35},
    latency_us=250,
)
```

```
result.fills_df()
result.orders_df()
result.equity_df()
result.metrics
```

11.2 Python strategy support

Optional: expose Strategy via a pybind11 trampoline for research convenience. Document that per-event Python callbacks incur C++/Python boundary and GIL overhead and are excluded from core performance benchmarks. Performance-sensitive strategies remain C++.

11.3 Experiment API

```
Experiment(
    strategy=ImbalanceStrategy,
    parameter_grid={
        "levels": [1, 5, 10],
        "entry_threshold": [0.2, 0.3, 0.4],
        "latency_us": [0, 100, 250, 500],
    },
).run()
```

12. Quantitative correctness and experiments

12.1 Walk-forward protocol

Tune parameters only on past data, freeze them, evaluate the next window, then advance. Never choose parameters using test-window performance.

```
Days 1-5  -> train / select parameters
Day 6     -> test
Days 2-6  -> train
Day 7     -> test
...       -> repeat
```

12.2 Strategy-visible information

Allowed at time t	Forbidden
Current/past book, trades, position, fills	Events after t
Current strategy state/configuration	Future midpoint / future volume
Past-derived features	Future fills or adverse-selection labels

12.3 Primary research question

How sensitive are short-horizon order-book imbalance strategies to execution latency, transaction costs and L2 queue-position assumptions?

Dimension	Example grid
Imbalance threshold	0.20 to 0.60
Latency	0, 50, 100, 250, 500, 1000 us
Queue model	Conservative / Neutral / Optimistic
Fees	Actual schedule or documented scenarios

Report gross vs net performance, fill rate, turnover, decision/arrival slippage and adverse selection. A strategy that disappears after realistic costs is a valid and useful result.

12.4 Secondary research question

How does latency alter spread capture, fill rate and adverse selection for passive market-making orders?

12.5 Metrics

Category	Metrics
PnL	Gross PnL, net PnL, fees/rebates, drawdown
Execution	Fill ratio, maker/taker %, spread captured, decision slippage, arrival slippage
Risk	Mean/max position, exposure, turnover
Selection	Adverse selection at 1/10/100/1000 ms
Returns	Sharpe/Sortino on fixed-time resampled returns; document annualisation

13. Performance engineering

13.1 Benchmark suite

- BM_AddOrder
- BM_CancelOrder
- BM_ModifyOrder
- BM_BestBidAsk
- BM_ApplyTrade
- BM_WalkBook
- BM_Replay1MEvents
- BM_Replay10MEvents

Use Release builds. Report ns/event and events/sec. Record CPU, RAM, OS, compiler/version, build flags and git commit. Use repeated runs/aggregate statistics rather than a single best sample.

13.2 Profiling

```
perf stat ./build/benchmarks/microback_bench
perf record ./build/apps/microback_replay ...
perf report
```

Track cycles, instructions, IPC, cache references/misses and branch misses. The optimisation loop is: correct -> benchmark -> profile -> change one bottleneck -> benchmark again -> document.

13.3 Natural concurrency

Do not make one instrument event stream multi-threaded merely for a CV keyword. Preserve deterministic sequential event semantics and parallelise independent parameter/data runs through a BatchRunner or Python multiprocessing.

13.4 Optional high-value optimisation study

Implement a second order-book backend only after profiling. Compare the simple std::map + std::list design against a cache-friendly price ladder / contiguous representation. Measure throughput, cache misses, cancellation performance and memory - then explain the trade-off between asymptotic complexity and hardware locality.

14. Testing and correctness

Test family	Minimum cases
Order book	Add bid/ask; best prices; FIFO; cancel first/middle/last; modify qty/price; remove empty level; partial/full fills; multi-level sweep
Accounting	Buy/sell round trip; partial position; realised/unrealised PnL; maker/taker fee/rebate
Simulator	Latency respected; no order before arrival; IOC residual cancel; crossing limit; passive queue; risk rejection
Causality	Construct price move between decision and arrival and prove stale book cannot fill the order
Determinism	Same run twice => identical result objects/files
Invariants	No negative quantities; remaining <= original; level totals match orders; filled + remaining == original; terminal statuses consistent

Run AddressSanitizer and UndefinedBehaviorSanitizer in CI; add ThreadSanitizer if parallel batch code becomes non-trivial. Optional stretch: fuzz the binary/event parser.

15. Repository, build and CI

15.1 Recommended repository

```
microback/
  CMakeLists.txt
  pyproject.toml
  README.md
  include/microback/   types.hpp event.hpp order_book.hpp replay_engine.hpp
                       execution.hpp portfolio.hpp risk.hpp strategy.hpp result.hpp
  src/                 order_book.cpp replay_engine.cpp execution.cpp portfolio.cpp
                       risk.cpp result.cpp binary_reader.cpp bindings.cpp
  python/microback/    experiment.py metrics.py plotting.py walkforward.py config.py
                       strategies/
  apps/                replay.cpp convert.cpp benchmark_run.cpp
  tests/cpp/           test_order_book.cpp test_execution.cpp test_portfolio.cpp ...
  tests/python/        test_bindings.py test_walkforward.py
  benchmarks/          book_benchmark.cpp replay_benchmark.cpp
  scripts/             collect_market_data.py convert_market_data.py profile.sh
  examples/            imbalance_strategy.py market_maker.py walk_forward.py
  docs/                architecture.md execution_model.md data_format.md
                       benchmarking.md assumptions.md
  .github/workflows/   ci.yml
```

15.2 Build targets

- microback_core
- microback_cli
- _microback (pybind11 extension)
- microback_tests
- microback_bench

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
ctest --test-dir build --output-on-failure
pip install -e .
```

15.3 CI jobs

- clang-format/style check
- GCC and Clang builds on Ubuntu
- C++ tests
- Python binding/walk-forward tests
- ASan/UBSan build
- package build
- benchmark smoke test only - no hosted-CI performance claims

16. Implementation sequence

Phase	Deliverable	Exit condition
1. Skeleton	CMake, types, tests, CI, synthetic events	Clean build/test on GCC + Clang
2. Order book	Correct L2/L3-capable book + invariants	All deterministic book tests pass
3. Replay	Canonical event source + event loop	Deterministic replay of fixtures
4. Execution	Market/limit/IOC, latency, partial fills	Causality and execution tests pass
5. Portfolio/risk	Accounting, fees, slippage, limits	PnL/risk tests pass
6. Strategies	Imbalance + market maker	Both run end-to-end
7. Real data	One exchange adapter/collector + validation	Reproducible real-market replay
8. Python	pybind11 + DataFrame results + plotting	pip install -e . and example notebook/script works
9. Research	Walk-forward experiments + latency/queue studies	Saved reproducible tables/plots
10. Performance	Google Benchmark + perf profiling	Reproducible benchmark report + one justified optimisation
11. Polish	README/docs/architecture/limitations	External reader can build/run/reproduce in minutes

17. Definition of Done

- C++20 limit-order-book implementation with deterministic event replay.
- Market and limit orders, GTC/IOC, partial fills and book walking.
- Configurable latency; maker/taker fees; L2 queue model; L3 mode if data permits.
- Portfolio/PnL accounting, risk limits, slippage and adverse-selection measurements.
- Order-book imbalance and inventory-aware market-making strategies.
- Real event dataset + synthetic fixtures + large synthetic benchmark stream.
- No-look-ahead and deterministic regression tests.
- Unit/integration tests + ASan/UBSan CI.
- Google Benchmark suite + Linux perf profile + documented optimisation.
- pybind11 C++/Python package with DataFrame outputs.
- Walk-forward experiment framework and reproducible research report.
- Clean README, architecture, execution assumptions, data format and benchmark docs.
- One-command build/test path and one-command demonstration run.

18. Evidence to capture for CV and interviews

Evidence	What to record
Throughput	Events/sec and ns/event for 1M/10M replay; hardware/compiler/build details.
Optimisation	Before/after benchmark for one profiler-identified bottleneck; state exactly what changed.
Execution realism	Latency sensitivity, fill rate, decision/arrival slippage and queue-model sensitivity.
Research hygiene	Walk-forward protocol and example where gross signal degrades after realistic costs.
Correctness	Number/scope of tests, deterministic result, sanitizer passes.
Architecture	Why C++ core + Python layer; why one stream stays sequential; why L2 queue is approximate.

Target CV entry - only fill measured placeholders

CV draft Market Microstructure & Backtesting Engine | C++20, Python, pybind11
- Built an event-driven C++20 limit-order-book simulator processing [X]M+ events/sec, supporting price-time priority, partial fills, configurable latency and L2/L3 queue-position modelling.
- Implemented maker/taker fees, book-walking slippage, portfolio/risk controls and walk-forward evaluation to prevent look-ahead bias.
- Exposed the C++ engine to Python via pybind11 and built reproducible experiments for latency sensitivity, order-book imbalance, fill quality and adverse selection.
- Profiled the hot path with Google Benchmark/Linux perf, improving [component] by [X]% through [specific optimisation].

45-second interview story

I built the project to be closer to quantitative research infrastructure than a pandas trading bot. The C++ engine reconstructs limit-order-book state from market events and simulates orders with latency, partial fills, transaction costs and queue-position assumptions. I exposed it to Python with pybind11 for walk-forward experiments and execution analysis. A key design choice is that L2 data cannot reveal exact queue position, so the engine explicitly compares conservative, neutral and optimistic models instead of assuming fills. I also benchmark and profile the replay path before optimising it, so I can explain both the market assumptions and the systems trade-offs.

19. Final guardrails

- Do not claim profitability as the primary outcome.
- Do not claim exact L2 queue position.
- Do not report throughput without reproducible hardware/compiler/build context.
- Do not use future data in strategy callbacks or parameter selection.
- Do not add concurrency where it breaks deterministic sequential semantics.
- Do not add web/cloud/UI features merely to increase technology count.
- Do not place unmeasured numbers on the CV. The specification is complete when it produces evidence, not buzzwords.

End state A reviewer should be able to see one coherent project spanning market microstructure, event-driven systems, C++ performance engineering, execution modelling, quantitative experimentation and Python research tooling - with every major claim backed by tests, benchmarks or reproducible experiments.